

datalab 报告

datalab 报告

姓名：杨泰熙

学号：2025201847

总分	bitAnd	bitXor	samesign	logtwo	byteSwap	reverse	logic
37/37	1/1	1/1	2/2	4/4	4/4	3/3	3/3

test 截图：

```
$ make && ./btest
gcc -O0 -Wall -std=gnu99 -lm -o btest bits.c btest.c decl.c tests.c
Make success.
bitAnd      1/1:    PASS
bitXor      1/1:    PASS
samesign    2/2:    PASS
logtwo      4/4:    PASS
byteSwap    4/4:    PASS
reverse     3/3:    PASS
logicalShift 3/3:    PASS
leftBitCount 4/4:    PASS
float_i2f   4/4:    PASS
floatScale2 4/4:    PASS
float64_f2i 3/3:    PASS
floatPower2 4/4:    PASS
Total points: 37/37
```

btest 全部通过

解题报告

亮点

- 1. float_i2f (向偶舍入 + 进位传播，踩坑最多)
- 2. leftBitCount (全 1 边界的补位技巧)
- 3. float64_f2i (53 位有效数拆两半移位)
- 4. floatScale2 (非规格化数左移自动规格化)
- 5. logicalShift (零警告的掩码构造)

float_i2f

```
unsigned float_i2f(int x) {
    unsigned m, e, frac, half, rest, sign;
```

```

if (x == 0)
    return 0;
m = x;
sign = 0;
if (x < 0) {
    m = ~m + 1;          /* 无符号运算取绝对值，INT_MIN 也安全 */
    sign = 0x80000000;
}
e = 31;
while (!(m >> e))
    e = e - 1;          /* 定位最高有效位 */
if (e > 23) {           /* 需要舍入 */
    frac = (m >> (e - 23)) & 0x7FFFFFFF;
    half = 1 << (e - 24); /* 被弃部分的"一半" */
    rest = m & (half + half - 1); /* 被弃的全部低位 */
    if (rest > half)      /* 超过一半：进位 */
        frac = frac + 1;
    if (rest == half)    /* 恰好一半：平局，向偶舍入 */
        if (frac & 1)
            frac = frac + 1;
} else {
    frac = (m << (23 - e)) & 0x7FFFFFFF; /* 精确表示，无需舍入 */
}
return sign + ((e + 127) << 23) + frac;
}

```

思路：求绝对值 → 找最高位 → 尾数截断并向偶舍入。两个关键点：

1. 向偶舍入用 rest（被弃低位）与 half（一半）比较实现：rest > half 进位；rest == half（平局）时尾数为奇数才进位。
2. 进位传播必须用 + 不能用 |：尾数 0x7FFFFFFF + 1 溢出成 0x800000 时要进位到阶码。INT_MAX → 2^{31} 时阶码 157 是奇数，其最低位恰好占据 bit 23，用 | 合并会吞掉进位，得到 2147483520.0 而非 2147483648.0。

leftBitCount

```

int leftBitCount(int x) {
    int v = ~x;          /* 左端连续1个数 = ~x 的前导零个数 */
    int r = 0;
    int t;
    t = !(v >> 16); r = r + (t << 4); v = v << (t << 4);
    t = !(v >> 24); r = r + (t << 3); v = v << (t << 3);
    t = !(v >> 28); r = r + (t << 2); v = v << (t << 2);
    t = !(v >> 30); r = r + (t << 1); v = v << (t << 1);
    t = !(v >> 31); r = r + t;
    t = !v;              /* 关键补丁：x = -1 时补到 32 */
    r = r + t;
    return r;
}

```

```
}
```

思路：二分法逐层判断“高 k 位是否全 0”（粒度 16/8/4/2/1）， $!(v \gg (32-k))$ 代替比较运算符，命中则累加并把 v 左移看下一层。易错点：16+8+4+2+1 最多累加到 31，而 `leftBitCount(-1)` 必须是 32，所以最后补 $!v$ 判断——走完 5 轮 v 仍为 0 说明原 v 全 0（即 x 全 1）。

float64_f2i

```
int float64_f2i(unsigned uf1, unsigned uf2) {
    int e = (uf2 >> 20) & 0x7FF;
    int sign = uf2 >> 31;
    unsigned mh = (uf2 & 0xFFFF) | 0x100000; /* 尾数高20位 + 隐含1 */

    unsigned mag;
    if (e >= 2047) return 0x80000000; /* NaN / 无穷 */
    if (e < 1023) return 0; /* |x| < 1, 向零舍入 */

    e = e - 1023;
    if (e > 30) return 0x80000000; /* 溢出 */
    if (e > 20) mag = (mh << (e - 20)) | (uf1 >> (52 - e));
    else mag = mh >> (20 - e); /* uf1 被全部移出 */
    if (sign) mag = -mag; /* 无符号取反即补码 */
    return mag;
}
```

思路：按阶码分档处理。不能定义 64 位变量，53 位有效数拆成“高 21 位 + $uf1$ ”两半移位合并。两个细节：① $e = 31$ 时“ $+2^{31}$ 溢出”、“ -2^{31} 恰好合法”、“更负的溢出”三种情况返回值都是 0x80000000，无需区分；② $e = 20$ 划进 `else` 分支（移位量 0），避免 $uf1 \gg 32$ 的未定义行为。

floatScale2

```
unsigned floatScale2(unsigned uf) {
    unsigned exp = (uf >> 23) & 0xFF;
    unsigned sign = uf & 0x80000000;
    if (exp == 0xFF) return uf; /* NaN 返回自身; 2*inf = inf */

    if (exp == 0) return sign | (uf << 1); /* 非规格化：尾数左移 */
    exp = exp + 1;
    if (exp == 0xFF) return sign | 0x7F800000; /* 溢出为无穷 */
    return (uf & 0x807FFFFF) | (exp << 23);
}
```

思路：按阶码分三类。最巧的一点是非规格化数直接左移：尾数最高位为 1 时左移后自然“顶”进指数域变成规格化数，IEEE 754 的设计保证了数学等价，无需任何特判。

logicalShift

```
int logicalShift(int x, int n) {
    return (x >> n) & ((0x7FFFFFFF >> n) | (1 << (31 + ~n + 1)));
}
```

思路：算术右移后用掩码清掉高位补进来的符号位。掩码 = $(0x7FFFFFFF \gg n) \mid (1 \ll (31 - n))$ ：低 $31 - n$ 位的 1 加上最高位的 1，恰好低 $32 - n$ 位全 1，且 $n = 0$ 时为全 1，不需要移位 32（无 UB）。特意不用 $\sim 0 \ll k$ 构造掩码——对负数左移会触发 `-Wshift-negative-value` 警告，题目要求零警告。

其余函数简述

- **bitXor**： $x \wedge y = \sim(x \& y) \& \sim(x \& y)$ （7 符号，恰好压线）。若分别构造“只有 x”和“只有 y”再合并需 8 符号超限，“整体排除”视角更省。
- **logtwo**：与 `leftBitCount` 同框架的二分定位，用 $(v > 0xFFFF) \ll 4$ 形式的条件移位代替 if。
- **byteSwap**：提取两个字节 → 掩码清位 → 交叉放回； $n = m$ 时同字节清掉再放回也正确。
- **reverse**：5 轮粒度翻倍的位反转（1/2/4/8/16 位），掩码依次为 `0x55555555`、`0x33333333`、`0x0F0F0F0F`、`0x00FF00FF`，最后两半直接互换。
- **samesign**： $!((x \gg 31) \wedge (y \gg 31))$ 判断符号位相同，再用 $!((!x) \wedge (!y))$ 排除“恰有一个为 0”（0 非正非负）。
- **bitAnd**：德摩根定律 $x \& y = \sim(\sim x \mid \sim y)$ 。

反馈/收获/感悟/总结

约 8 小时。最难的是 `float_i2f` 的向偶舍入和 `float64_f2i` 的溢出边界。收获最大的两个坑：① 浮点尾数进位必须用 + 让进位传播到阶码，用 | 会静默吞掉；② 无符号运算天然环绕，是处理 `INT_MIN` 的利器。另外体会到 IEEE 754 设计的自治性——非规格化数左移自动规格化、 2^x 恰好是尾数中的单个 1，都能省去大量特判代码。

参考的重要资料

1. Bit Twiddling Hacks（前导零二分、位反转掩码）：
<https://graphics.stanford.edu/~seander/bithacks.html>
2. CS:APP 第 2 章（信息的表示和处理，IEEE 754 浮点格式）
3. IEEE 754 单精度浮点格式说明：https://en.wikipedia.org/wiki/Single-precision_floating-point_format